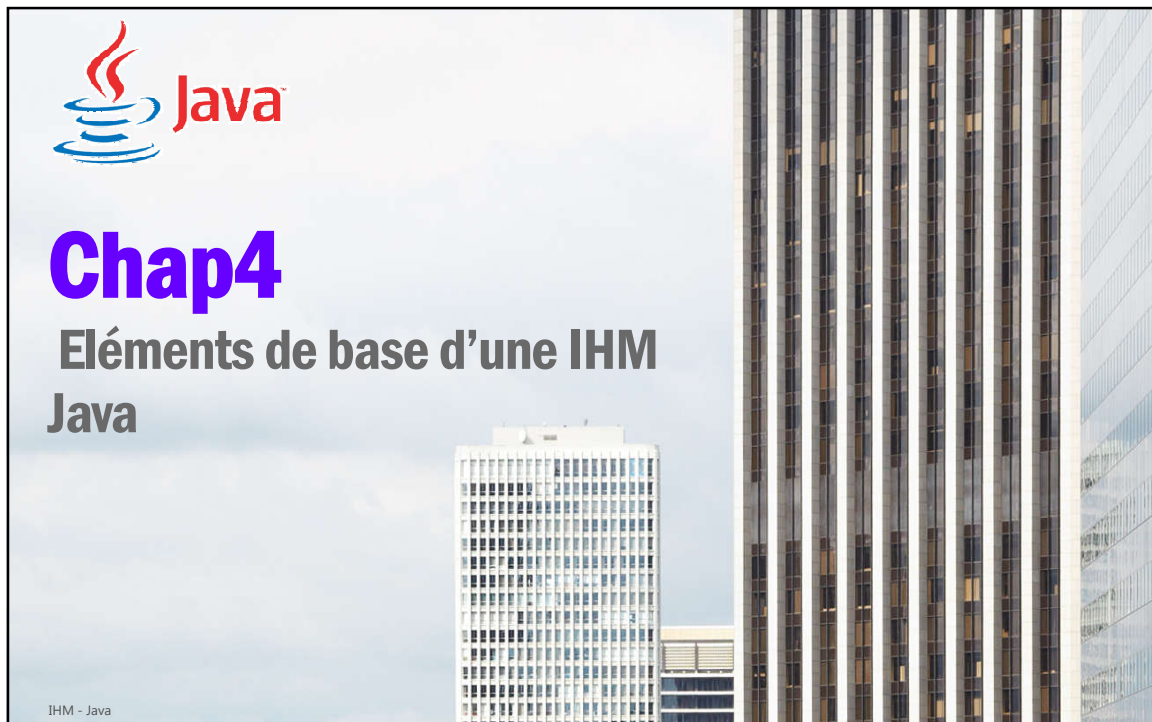




Plan du cours



Chap 1 : Initiation au Langage Java

- Introduction
- Outils de développement
- Données, Types et Operateurs
- Les structures alternatives et répétitives
- Les tableaux de données et chaînes de caractères
- Les instructions d'écriture et de lecture

Chap 2 : Java Orienté Objet

- Classes et Objets
- Héritage
- Packages
- Classes internes et Interfaces
- Les exceptions

Chap 3 : Introduction aux IHMs

- Evolution des IHM dans le temps
- Ergonomie
- Le patron d'architecture logicielle MVC

Chap 4 : Eléments de base d'une IHM Java

- Composants de base
- Composants SWING
- Les conteneurs
- Les composants atomiques
- Evènements & Ecouteurs
- Principes programmation événementielle
- Listeners java
- Gestionnaires de positionnement

Chap 5 : Connexion à une base de données

IHM - Java

2

Éléments de base d'une IHM Java

(CHAPITRE 4)



Java et les interfaces graphiques:

Une IG : intermédiaire entre l'utilisateur et la partie «métier» d'un logiciel.

- ce qui relève de l'IG
 - gérer le fait que le bouton soit actif ou non
 - commande "Enregistrer"
 - lancer la sauvegarde quand on clique
- ce qui ne relève pas de l'IG
 - la sauvegarde elle-même!



Éléments de base d'une IHM Java

(CHAPITRE 4)



Définition : une API :

- Une Interface de Programmation d'Application (**API**) est un ensemble de classes regroupées par leur proximité fonctionnelle.
- En java, elle se présente sous la forme d'un **paquetage** (package) ou d'un ensemble de paquetages
- Pour concevoir une IHM, Java offre deux API principales ;
 - **java.awt** (Abstract WindowToolkit) est l'ancienne, à n'utiliser que quand on n'a pas d'équivalent dans la nouvelle
 - **javax.swing** doit être utilisée prioritairement

N.B. Le mélange inconsidéré des deux peut provoquer des erreurs impossibles à identifier

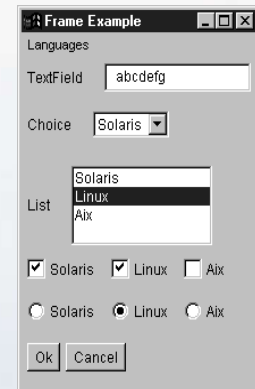
Éléments de base d'une IHM Java

(CHAPITRE 4)



Définition : une API :

- Classe de base des composants **awt** : **Component**.
- Classe de base des composants **swing** : **JComponent**.
- On distingue, par service :
 - les classes des **conteneurs**
 - les classes d'**interaction**
 - Les **menus** et **dialogues**
- Les programmes à interfaces graphiques sont dirigés par évènements.



IHM - Java

5

Éléments de base d'une IHM Java

(CHAPITRE 4)



La notion de composant :

- De façon générale, un **composant** est un élément graphique
- bout de code réutilisable.

Composants légers vs composants lourds

- On parle de **composant lourd** lorsqu'on utilise l'API du système hôte pour les créer, ce que fait **AWT**.
- Un **composant léger**, comme en **Swing**, est créé de toute pièce par l'API Java sans faire appel à son correspondant du système hôte.
- Cela permet d'avoir des applications qui ont exactement le même «look» quelle que soit la machine sur laquelle on la tourne

IHM - Java

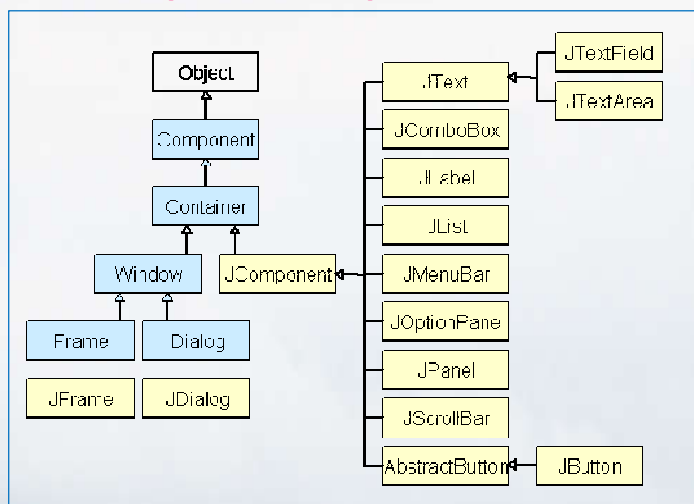
6

Éléments de base d'une IHM Java

(CHAPITRE 4)



Les classes mères : Component et JComponent :



IHM - Java

7

Éléments de base d'une IHM Java

(CHAPITRE 4)



Relation entre les composants:

- Tous les composants sont hébergés par un conteneur et toute application graphique doit utiliser un conteneur primaire (composant lourd):
 - les applets **JApplet**
 - les fenêtres **JFrame**, **JWindow**
 - les boîtes de dialogues **JDialog**

Il est possible de connaître le conteneur d'un composant en utilisant la méthode **getParent**.

IHM - Java

8

Éléments de base d'une IHM Java

(CHAPITRE 4)



Les conteneurs primaires :

Les quatre conteneurs primaires sont construits sur le même modèle en couches.

- Couche supérieure : **GlassPane** un panneau transparent pour la gestion des événements
- Sous ce panneau se situe le **contentPane** qui accueille les composants graphiques. Par défaut c'est un **JPanel**.
- Il est contenu par le **LayeredPane** qui peut être utilisé pour empiler des composants à des "profondeurs" différentes.
- Chaque conteneur contient aussi une barre de menu **menuBar**,
- Tous ces éléments sont contenus dans un élément principal de type **JRootPane**

IHM - Java

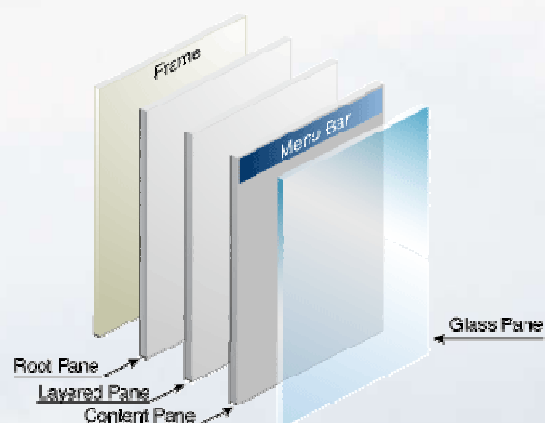
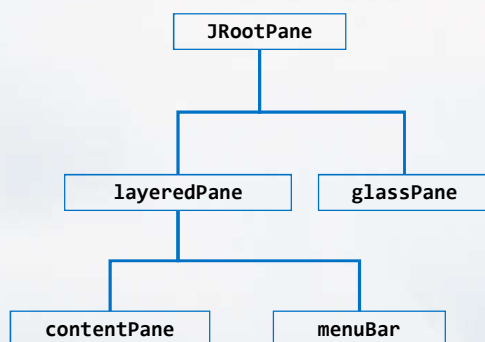
9

Éléments de base d'une IHM Java

(CHAPITRE 4)



Les conteneurs primaires :



IHM - Java

10

Éléments de base d'une IHM Java

(CHAPITRE 4)



Accès aux panneaux :

La classe **JFrame** contient les méthodes d'accès aux panneaux :

- **getRootPane()**
- **getLayeredPane()**
- **getContentPane()**
- **getGlassPane()**

Ajout de composant :

- Les composants (boutons, labels,...) sont placés sur le **contentPane**. Par défaut ce conteneur est un **JPanel**. Il est possible d'accéder à ce conteneur avec la méthode **getContentPane**, puis d'appeler la méthode **add** pour ajouter des composants. Cette méthode accepte en paramètre un objet de type **JComponent**.

```
JFrame fen = new JFrame () ;
Container cont = fen.getContentPane();
cont.add(myComponent);
```

IHM - Java

11

Éléments de base d'une IHM Java

(CHAPITRE 4)



Les fenêtres : JWindow, JFrame :

Swing propose deux types de fenêtres : **JWindow** et **JFrame**. Ces deux composants descendent de **Window** (composant awt).

Méthode communes

- Pour créer une fenêtre, le constructeur est appelé. Par défaut, une fenêtre qui est créée mais pas affichée. Pour l'afficher, il faut faire appel à la méthode **setVisible**.

```
JFrame fen = new JFrame() ; // identique pour JWindow
fen.setVisible( true ) ;
```

IHM - Java

12

Éléments de base d'une IHM Java

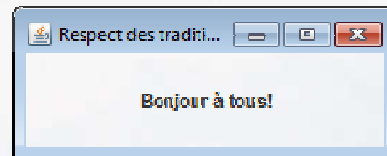
(CHAPITRE 4)



Les fenêtre JFrame:

```
import java.awt.Color;
import java.awt.Container;
import javax.swing.JFrame;
import javax.swing.JLabel;

public class Test {
    public static void main(String[] args) {
        JFrame cadre = new JFrame("Respect des traditions");
        Container c= cadre.getContentPane() ;
        JLabel etiquette = new JLabel("Bonjour à tous!",JLabel.CENTER);
        c.add(etiquette);
        c.setBackground(Color.WHITE);
        cadre.setSize(250, 100);
        cadre.setVisible(true);
    }
}
```



IHM - Java

13

Éléments de base d'une IHM Java

(CHAPITRE 4)



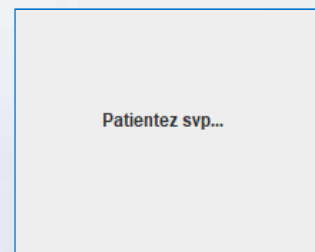
Différences entre JFrame et JWindow :

La classe **JWindow** permet de créer une fenêtre sans aucune bordure et aucun bouton.

Elle ne peut être fermée que par le programme qui l'a construite (ou en mettant fin à l'application via le système d'exploitation en tuant le processus associé).

D'une manière générale, **JWindow** n'est utilisée que pour créer des fenêtres particulières comme des écrans d'attente (SplashScreen).

```
import javax.swing.JLabel;
import javax.swing.JWindow ;
public class Test{
    public static void main(String []args){
        JWindow win = new JWindow () ;
        JLabel etiquette =
            new JLabel("Patientez svp...", JLabel.CENTER);
        win.getContentPane().add(etiquette);
        win.setSize(300,300) ;
        win.setLocation(500,200) ;
        win.setVisible(true) ;
    }
}
```



IHM - Java

14

Eléments de base d'une IHM Java

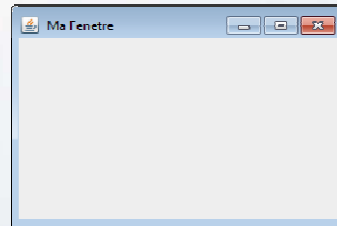
(CHAPITRE 4)



Classe de type JFrame :

```
import javax.swing.*;
class Fenetre extends JFrame {
    public Fenetre () {
        setTitle(" Ma Fenetre ");
        setBounds(100, 100, 300, 200);
        setVisible(true);
    }
}

public class Test{
    public static void main(String[] args) {
        new Fenetre ();
    }
}
```



IHM - Java

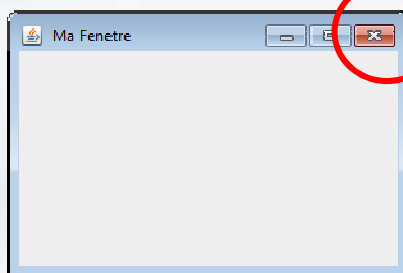
15

Eléments de base d'une IHM Java

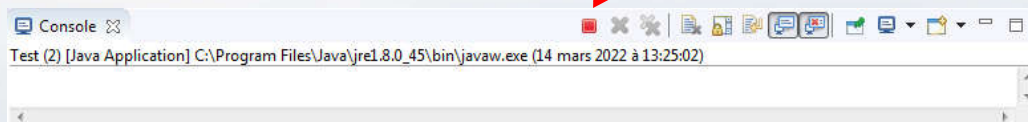
(CHAPITRE 4)



Classe de type JFrame :



La fermeture de la fenêtre ne termine pas le programme



IHM - Java

16

Éléments de base d'une IHM Java

(CHAPITRE 4)



Fermeture d'une JFrame :

```
import javax.swing.*;
class Fenetre extends JFrame {
    public Fenetre () {
        setTitle(" Ma Fenetre ");
        setBounds(100, 100, 300, 200);
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        setVisible(true);
    }
}
public class Test{
    public static void main(String[] args) {
        new Fenetre ();
    }
}
```



IHM - Java

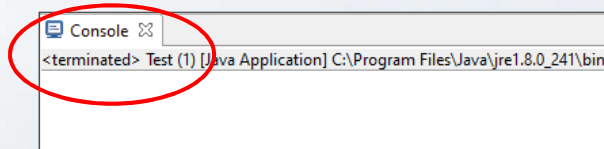
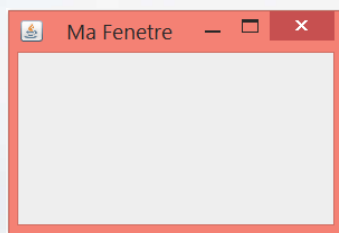
17

Éléments de base d'une IHM Java

(CHAPITRE 4)



Fermeture d'une JFrame :



IHM - Java

18

Éléments de base d'une IHM Java

(CHAPITRE 4)



Les boîtes de dialogues (DialogBox) :

- **Swing** propose des boîtes de dialogues afin de communiquer rapidement avec l'utilisateur. La classe **JOptionPane** permet de créer des boîtes de dialogues génériques mais aussi de les modifier en fonction des besoins de l'application.
- Les méthodes d'affichage sont **statiques** (pas nécessaire d'instancier une classe pour les utiliser).

Les dialogues de message

Les plus simples, elles informent l'utilisateur en affichant un texte simple et un bouton de confirmation. Elles restent affichées tant que l'utilisateur n'a pas cliqué sur le bouton et bloquent l'application en cours.

Pour afficher un message on utilise la méthode **showMessageDialog** de la classe **JOptionPane**. Cette méthode propose de nombreuses options accessibles via la surcharge.



Éléments de base d'une IHM Java

(CHAPITRE 4)



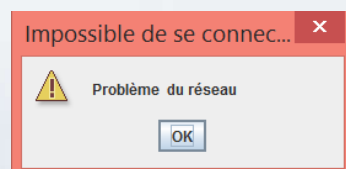
Les boîtes de dialogues (DialogBox) :

SIMPLE BOITE

```
JOptionPane.showMessageDialog(fen, "Un message");
```

BOITE AVERTISSEMENT

```
JOptionPane.showMessageDialog(fen, "Problème de  
réseau", "Impossible de se connecter",  
JOptionPane.WARNING_MESSAGE);
```



Eléments de base d'une IHM Java

(CHAPITRE 4)

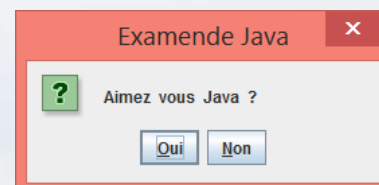
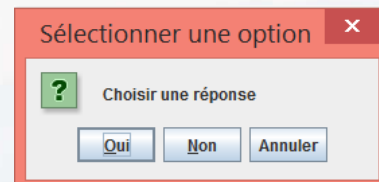


Les boîtes de dialogues (DialogBox) :

BOITE CONFIRMATION/QUESTION

```
JOptionPane.showConfirmDialog(fen, "Choisir une réponse");
```

```
JOptionPane.showConfirmDialog(fen, "Aimez vous Java ?", "Examen de Java",  
JOptionPane.YES_NO_OPTION);
```



IHM - Java

21

Eléments de base d'une IHM Java

(CHAPITRE 4)



Les boîtes de dialogues (DialogBox) :

La boîte de dialogue renvoie un entier en fonction du choix de l'utilisateur. Cette valeur peut être utilisée pour modifier le comportement du programme. Cet entier est défini comme une constante de la classe **JOptionPane**.

Par exemple pour le cas précédent :

```
int rep = JOptionPane.showConfirmDialog(fen, "Aimez-vous Java ?", "Examen de  
Java", JOptionPane.YES_NO_OPTION) ;  
if ( rep == JOptionPane.YES_OPTION) {  
    // ...  
}  
if ( rep == JOptionPane.NO_OPTION) {  
    // ...  
}
```

IHM - Java

22

Eléments de base d'une IHM Java

(CHAPITRE 4)

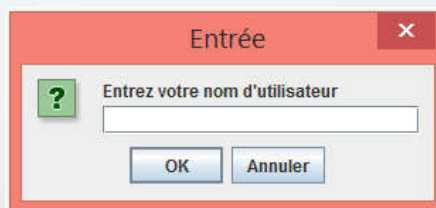


Les boîtes de dialogues (DialogBox) :

LA BOITE DE SAISIE

- La méthode **showInputDialog** affiche une boîte de dialogue comprenant une entrée de saisie (**JTextField**) en plus des boutons de validation. Après validation elle retourne une chaîne de caractères (**String**) si l'utilisateur a cliqué sur OK, sinon elle renvoie **null**

```
String rep = JOptionPane.showInputDialog( fen, "Entrez votre nom d'utilisateur" );
```



IHM - Java

23

Eléments de base d'une IHM Java

(CHAPITRE 4)

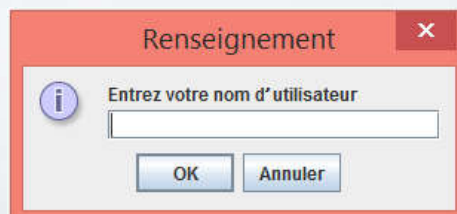


Les boîtes de dialogues (DialogBox) :

LA BOITE DE SAISIE

La méthode **showInputDialog** est surchargée afin de pouvoir modifier l'aspect de la boîte de dialogue. Pour changer le titre de la boîte on va utiliser le code suivant :

```
String rep = JOptionPane.showInputDialog(fen, "Entrez votre nom d'utilisateur",  
"Renseignement", JOptionPane.INFORMATION_MESSAGE);
```



IHM - Java

24

Eléments de base d'une IHM Java

(CHAPITRE 4)



Boite de dialogue personnalisée :

Pour construire des boîtes de dialogue plus complexes on utilise la méthode **showOptionDialog**. Cette méthode admet **8** paramètres (certains pouvant être à **null**). Elle renvoie un entier qui correspond au bouton qui a été cliqué.

Le prototype est le suivant :

```
showOptionDialog(Component parentComponent, Object message, String title,
int optionType, int messageType, Icon icon, Object[] options, Object
initialValue)
```

IHM - Java

25

Eléments de base d'une IHM Java

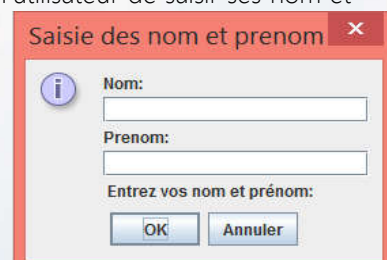
(CHAPITRE 4)



Boite de dialogue personnalisée :

Exemple : Construire une boîte de dialogue qui demande à l'utilisateur de saisir ses nom et prénom. La boîte contient 3 labels et 2 champs de saisie.

```
Fenetre fen=new Fenetre ();
JLabel labelNom = new JLabel("Nom:");
JLabel labelPrenom = new JLabel("Prenom:");
JTextField nom = new JTextField();
JTextField prenom = new JTextField();
JLabel lab = new JLabel("Entrez vos nom et prénom:");
Object [] tab = new Object []{ labelNom , nom , labelPrenom , prenom ,lab };
int rep = JOptionPane.showOptionDialog(fen,tab,"Saisie des nom et
prenom",JOptionPane.OK_CANCEL_OPTION,JOptionPane.INFORMATION_MESSAGE,null,
null);
```



IHM - Java

26

Éléments de base d'une IHM Java

(CHAPITRE 4)



Le panneau JPanel :

- **Exemple** : Panneau comportant plusieurs composants : trois **JLabel**, deux **JTextField** et un **JTextArea** (formulaire de renseignements simple).

```
class notrePanneau extends JPanel{
    private JLabel labelNom ;private JTextField nom ;
    private JTextField prenom ;private JLabel labelPrenom;
    private JTextArea adresse ;private JLabel labelAdresse;
    public notrePanneau() {
        super () ;
        labelNom = new JLabel ( " Nom : " ) ;
        labelPrenom = new JLabel(" Prenom : " ) ;
        labelAdresse = new JLabel("Adresse :") ;
        nom = new JTextField(5) ;
        prenom = new JTextField(5) ;
        adresse = new JTextArea("",3,10) ;
        this.add(labelNom);this.add(nom);
        this.add(labelPrenom);this.add(prenom );
        this.add(labelAdresse);this.add(adresse);
    }
}
```

IHM - Java

27

Éléments de base d'une IHM Java

(CHAPITRE 4)



Le panneau JPanel :

```
public class TestPanel{
    public static void main ( String [] args ) {
        JFrame fen = new JFrame () ;
        notrePanneau P = new notrePanneau();
        fen.setContentPane(P);
        fen.pack() ;
        fen.setVisible(true) ;
        fen.setTitle("Renseignements") ;
    }
}
```

IHM - Java

28

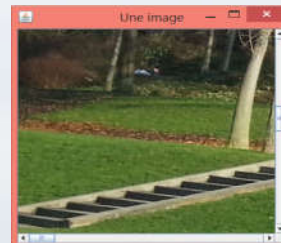
Éléments de base d'une IHM Java

(CHAPITRE 4)



Le panneau à défilement : JScrollPane:

```
public class TestPanel{
    public static void main ( String [] args ) {
        JFrame fra = new JFrame () ;
        JLabel monImage = new JLabel ( new ImageIcon("E:\\image1.jpg"));
        JScrollPane lePanneau = new JScrollPane(monImage) ;
        fra.setContentPane(lePanneau) ;
        fra.setTitle( " Une image " ) ;
        fra.setSize(400 ,400) ;
        fra.setDefaultCloseOperation( JFrame.EXIT_ON_CLOSE);
        fra.setResizable(true);
        fra.setVisible(true) ;
    }
}
```



IHM - Java

29

Éléments de base d'une IHM Java

(CHAPITRE 4)



Le panneau à défilement : JScrollPane:

Les applications traitant du texte ou des images n'affichent souvent qu'une partie du document pour éviter de monopoliser toute la surface d'affichage.

Des glissières sont affichées sur les côtés afin de pouvoir se déplacer dans le document. Le composant **JScrollPane** permet d'implémenter cette fonction.



IHM - Java

30

Eléments de base d'une IHM Java

(CHAPITRE 4)



Exercice :

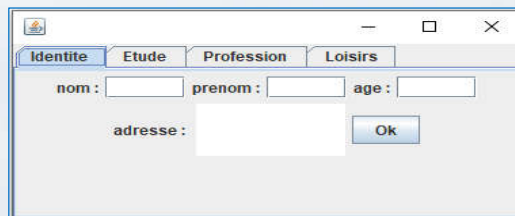
Ecrire un programme qui demande à l'utilisateur son nom via une boîte de dialogue.

Si le nom saisi est « Alami Mohamed » alors le programme :

1. Affiche une boîte de dialogue confirmant son enregistrement
2. Affiche la fenêtre correspondant à la figure ci-dessous (**JTabbedPane**).

Sinon le programme :

1. Affiche une boîte d'avertissement que le nom saisi n'est pas enregistré
2. Se termine.



IHM - Java

31

Eléments de base d'une IHM Java

(CHAPITRE 4)



Le panneau divisé : JSplitPane :

Séparer une interface en deux volets est utilisé pour mettre en vis-à-vis deux documents, ou un document et une barre d'outils. La séparation peut être horizontale ou verticale selon les interfaces. Ce type d'interface fait appel au **JSplitPane**.



IHM - Java

32

Éléments de base d'une IHM Java

(CHAPITRE 4)



Le panneau divisé : JSplitPane :

```
public class TestPanel{
    public static void main ( String [] args ) {
        JFrame fra = new JFrame () ;
        JTextArea source = new JTextArea() ;
        JTextArea traduction = new JTextArea() ;
        JSplitPane lePanneau=new JSplitPane( JSplitPane.HORIZONTAL_SPLIT,
        source,traduction) ;
        fra.setContentPane(lePanneau) ;
        fra.setSize(400 ,400) ;
        fra.setDefaultCloseOperation( JFrame.EXIT_ON_CLOSE);
        fra.setVisible(true) ;
    }
}
```



IHM - Java

33

Éléments de base d'une IHM Java

(CHAPITRE 4)

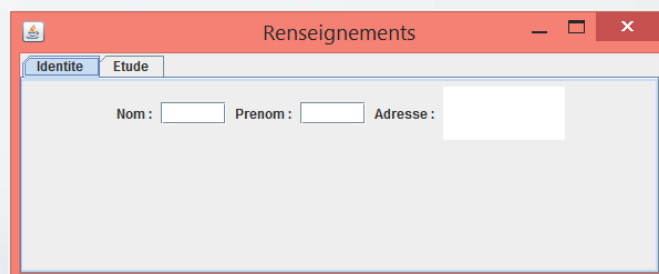


Le panneau à onglets : JTabbedPane :

JTabbedPane permet de construire des interfaces en utilisant des onglets. Les composants sont regroupés de manière thématique. La méthode **addTab** permet d'ajouter un composant dans une nouvelle feuille. Généralement on utilise un **JPanel**.

Exemple : Application à deux onglets

Deux **JPanel** dans un **JTabbedPane**



IHM - Java

34

Éléments de base d'une IHM Java

(CHAPITRE 4)



Le panneau à onglets : JTabbedPane :

L'onglet Identite

```
public class FicheIdentite extends JPanel{
    private JLabel labelNom ;    private JTextField nom ;
    private JTextField prenom ; private JLabel labelPrenom;
    private JTextArea adresse ; private JLabel labelAdresse;
    public FicheIdentite() {
        super () ;
        labelNom = new JLabel ( " Nom : " ) ;
        labelPrenom = new JLabel ( " Prenom : " ) ;
        labelAdresse = new JLabel ( " Adresse : " ) ;
        nom = new JTextField(5) ;
        prenom = new JTextField(5) ;
        adresse = new JTextArea("",3,10) ;
        this.add(labelNom);this.add(nom);
        this.add(labelPrenom);this.add(prenom) ;
        this.add(labelAdresse);this.add(adresse); }
}
```

IHM - Java

35

Éléments de base d'une IHM Java

(CHAPITRE 4)



Le panneau à onglets : JTabbedPane :

L'onglet Etude

```
public class FicheEtude extends JPanel{
    private JLabel Etude ;
    private JTextField etude ;
    private JLabel Resume;
    private JTextArea resume;
    public FicheEtude() {
        super () ;
        Etude = new JLabel ( " Etude : " ) ;
        Resume = new JLabel ( " Résumé : " ) ;
        etude = new JTextField(10) ;
        resume = new JTextArea("",10,30) ;
        this.add(Etude);
        this.add(etude);
        this.add(Resume);
        this.add(resume ); }
}
```

IHM - Java

36

Éléments de base d'une IHM Java

(CHAPITRE 4)

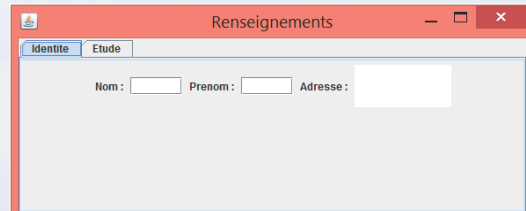


Le panneau à onglets : JTabbedPane :

Le menu

```
class MenuOnglets extends JTabbedPane
{
    private FicheIdentite i ;
    private FicheEtude e ;
    public MenuOnglets(){
        super() ;
        i = new FicheIdentite() ;
        e = new FicheEtude() ;
        this.addTab("Identite ",i);
        this.addTab(" Etude ",e) ;
    }
}
```

```
public class TestPanel{
    public static void main(String[] args){
        JFrame fen = new JFrame () ;
        MenuOnglets m= new MenuOnglets();
        fen.setContentPane(m);
        fen.pack () ;
        fen.setVisible(true) ;
        fen.setTitle("Renseignements");
    }
}
```



IHM - Java

Éléments de base d'une IHM Java

(CHAPITRE 4)



Les labels : JLabel :

Les composants atomiques sont tous les composants élémentaire de Swing. Ce sont les boutons, les labels, les menus,...

Un label est une simple chaîne de caractères informative (il peut aussi contenir une image). Pour créer un nouveau label il suffit d'appeler le constructeur **JLabel**.

LABEL TEXTE

```
JLabel monLabel = new JLabel ("Nom : ") ;
```

Ce label est ajouté à un conteneur avec la méthode **add()**.

LABEL IMAGE

```
ImageIcon img = new ImageIcon("arbre.gif") ;
JLabel unLabel2 = new JLabel ( img ) ;
```

IHM - Java

38

Éléments de base d'une IHM Java

(CHAPITRE 4)

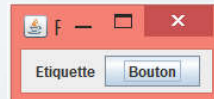


Les boutons : JButton:

Les boutons sont les composants les plus utilisés pour interagir avec l'utilisateur. Swing propose plusieurs types de boutons. Tous les boutons héritent de la classe abstraite **AbstractButton**.

Le composant JButton

Le bouton le plus utilisé est le **JButton**. Il crée un bouton qui peut être cliqué par l'utilisateur à l'aide de la souris. Généralement le texte affiché dans le bouton est passé comme paramètre au constructeur. Toutefois, il est possible de le modifier à l'aide de la méthode **setText**.



IHM - Java

39

Éléments de base d'une IHM Java

(CHAPITRE 4)



Les boutons : JButton:

```
public class TestPanel{
    public static void main ( String [] args ) {
        JFrame fen = new JFrame () ;
        JPanel pan = new JPanel () ;
        JLabel unLabel = new JLabel ( " Fermer" ) ;
        JButton unBouton = new JButton ( " Oui" ) ;
        pan.add(unLabel);
        pan.add(unBouton);
        fen.setContentPane(pan);
        fen.pack () ;
        fen.setVisible(true) ;
        fen.setTitle("Renseignements") ;
    }
}
```



IHM - Java

40

Eléments de base d'une IHM Java

(CHAPITRE 4)



Le composant JToggleButton :

Swing propose un type de bouton particulier, les boutons à bascule : **JToggleButton**. Ces boutons conservent leur état après le relâchement de la souris. Ils sont utilisés pour représenter un état booléen (comme par exemple l'effet souligné dans un traitement de texte). On peut comparer les **Jbutton** aux boutons poussoirs et les **JToggleButton** aux interrupteurs.

```
JToggleButton unBouton = new JToggleButton ("Un Bouton toggle");
```



IHM - Java

41

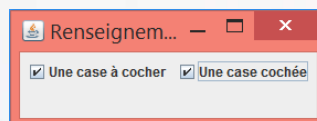
Eléments de base d'une IHM Java

(CHAPITRE 4)



Les cases à cocher : JCheckBox :

Les cases à cocher permettent de matérialiser des choix binaires d'une manière plus usuelle que les boutons à bascules (**JToggleButton**). Comme les boutons, elles héritent de la classe abstraite **AbstractButton**.



Comme pour les boutons à bascule, on lit l'état d'une case à cocher à l'aide de la méthode **isSelected()** qui renvoie une valeur booléenne et on la modifie avec **setSelected**.

```
if (maCaseACocher.isSelected()== true) {  
    // la case est cochée  
}  
else {  
    // la case n'est pas cochée  
}
```

IHM - Java

42

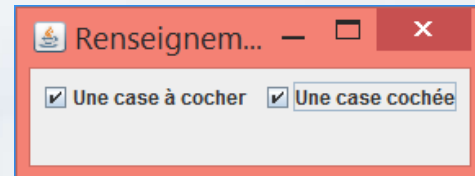
Éléments de base d'une IHM Java

(CHAPITRE 4)



Les cases à cocher : JCheckBox :

```
public class TestPanel{
    public static void main ( String [] args ) {
        JFrame fen = new JFrame () ;
        JPanel pan = new JPanel () ;
        JCheckBox cb1 = new JCheckBox("Une case à cocher");
        JCheckBox cb2 = new JCheckBox("Une case cochée",true);
        pan.add(cb1);
        pan.add(cb2);
        fen.setContentPane(pan);
        fen.pack () ;
        fen.setVisible(true) ;
        fen.setTitle("Renseignements") ;}
}
```



IHM - Java

43

Éléments de base d'une IHM Java

(CHAPITRE 4)

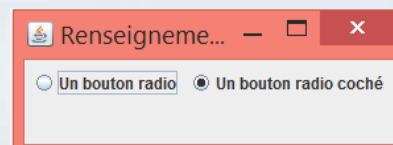


Les cases à cocher : JCheckBox :

Les boutons radio **JRadioButton** sont des boutons à choix exclusif, il permettent de choisir un (et un seul) élément parmi un ensemble. Comme les **AbstractButton** dont ils héritent, ils ont deux états. Pour créer un bouton radio, on peut utiliser différentes surcharges du constructeur. L'une des plus utilisée permet d'initialiser la variable de texte associée au bouton : **JRadioButton(String)**.

Il est possible de choisir l'état du bouton de la création avec le constructeur **JRadioButton(String, boolean)**.

```
JRadioButton bouton1 = new JRadioButton("Un bouton radio");
JRadioButton bouton2=new JRadioButton("Un bouton radio coché",true);
```



IHM - Java

44

Éléments de base d'une IHM Java

(CHAPITRE 4)



Les cases à cocher : JCheckBox :

Les boutons radio doivent être regroupés dans un **BoutonGroup** pour avoir un comportement exclusif. Il s'agit d'un groupe logique où un et seul bouton peut être actif (**true**). Le **BoutonGroup** n'a aucune incidence sur l'affichage.

```
JRadioButton b1 , b2, b3;
ButtonGroup B=null;

b1= new JRadioButton(" b1");
b2= new JRadioButton(" b2");
b3= new JRadioButton(" b3");

B= new ButtonGroup();
B.add( b1);
B.add( b2);
B.add( b3);
```

IHM - Java

45

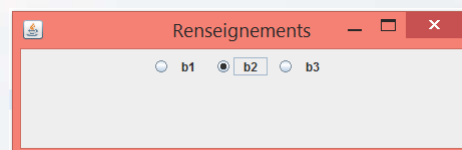
Éléments de base d'une IHM Java

(CHAPITRE 4)



Les cases à cocher : JCheckBox :

```
public class TestPanel{
    public static void main ( String [] args ) {
        JFrame fen = new JFrame () ;
        JPanel pan = new JPanel () ;
        JRadioButton b1 , b2, b3;
        ButtonGroup B=null;
        b1= new JRadioButton(" b1 ");
        b2= new JRadioButton(" b2 ");
        b3= new JRadioButton(" b3 ");
        B= new ButtonGroup();
        B.add(b1);B.add(b2);B.add(b3);
        pan.add( b1);pan.add( b2);pan.add( b3);
        fen.setContentPane(pan);
        fen.setVisible(true) ;
        fen.setTitle("Renseignements") ;
    }
}
```



IHM - Java

46

Éléments de base d'une IHM Java

(CHAPITRE 4)



Les listes de choix : JList :

Ce composant permet de choisir un (ou plusieurs) élément(s) parmi un ensemble prédéfini. Cet ensemble peut être un tableau ou vecteur d'objets quelconques. Les éléments sont présentés sous la forme d'une liste dans laquelle les éléments choisis sont surlignés.



IHM - Java

47

Éléments de base d'une IHM Java

(CHAPITRE 4)



Les listes de choix : JList :

```
public class Test{
    public static void main( String [] args) {
        JPanel pan=new JPanel();
        JLabel l =new JLabel("Chisissez un instrument");
        String [] lesElements={"Guitare", "Basse", "Clavier", "Batterie ",
        "Percussions", "Flute", "Violon"};
        JList instruments = new JList( lesElements);
        pan.add(l);
        pan.add(instruments);
        JFrame fen = new JFrame ("Liste Musique");
        fen.setContentPane( pan );
        fen.pack();
        fen.setVisible( true);
    }
}
```



IHM - Java

48

Éléments de base d'une IHM Java

(CHAPITRE 4)



Les boîtes combo : JComboBox :

Les boîtes combo permettent de choisir un seul élément parmi une liste proposée. On les utilise quand l'ensemble des éléments à afficher n'est pas connu lors de la conception. Comme les listes de choix, on peut les construire en passant un tableau d'objet en paramètre de construction

```
JComboBox instruments = new JComboBox( lesElements);
```

Les méthodes `getSelectedIndex` et `getSelectedItem` permettent de connaître l'indice et l'objet sélectionné.



IHM - Java

49

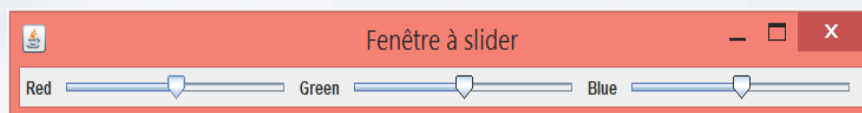
Éléments de base d'une IHM Java

(CHAPITRE 4)



Les glissières : JSlider :

Les glissières permettent de proposer à l'utilisateur une interface de saisie plus intuitive qu'un champ de texte pour régler certains paramètres. Swing propose le composant `JSlider` pour représenter un réglage variable.



IHM - Java

50

Eléments de base d'une IHM Java

(CHAPITRE 4)

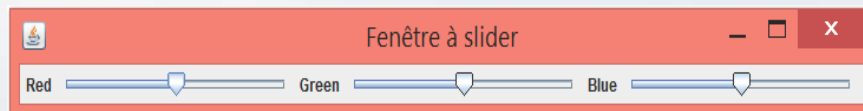


Les glissières : JSlider :

```

JPanel pan=new JPanel();
JLabel b=new JLabel("Blue");
JLabel r=new JLabel("Red");
JLabel g=new JLabel("Green");
JSlider sBlue = new JSlider ();
JSlider sRed = new JSlider ();
JSlider sGreen = new JSlider ();
pan.add(r);pan.add(sRed);
pan.add(g);pan.add(sGreen );
pan.add(b);pan.add(sBlue);

```



IHM - Java

51

Eléments de base d'une IHM Java

(CHAPITRE 4)

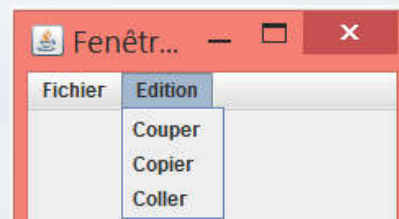


Les menus :

JMenuBar, JMenu, JMenuItem, JRadioButtonMenuItem, JCheckBoxMenuItem :

Les barres de menus permettent de regrouper de nombreuses fonctions d'une manière ergonomique.

On utilise un composant **JMenuBar** pour construire une barre de menus. Les menus sont construits à partir de la classe **JMenu**. Ils sont placés dans la **JMenuBar** avec la méthode **add**. Ces menus sont constitués d'éléments appartenant à la classe **JMenuItem** ou à l'une de ses classes filles (**JRadioButtonMenuItem**, **JCheckBoxMenuItem**). Les éléments de menus sont ajoutés aux menus à l'aide de la méthode **add**.



IHM - Java

52

Eléments de base d'une IHM Java

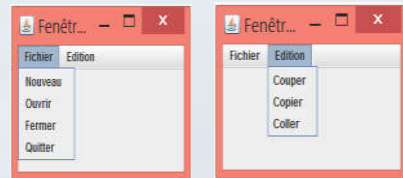
(CHAPITRE 4)



Les menus :

JMenuBar, JMenu, JMenuItem, JRadioButtonMenuItem, JCheckBoxMenuItem :

```
JFrame fen = new JFrame ("Fenêtre à menus");
JMenu menuFichier = new JMenu(" Fichier ");
JMenuItem Nouveau = new JMenuItem(" Nouveau "); menuFichier.add ( Nouveau);
JMenuItem Ouvrir = new JMenuItem(" Ouvrir "); menuFichier.add ( Ouvrir);
JMenuItem Fermer = new JMenuItem(" Fermer "); menuFichier.add ( Fermer);
JMenuItem Quitter = new JMenuItem(" Quitter "); menuFichier.add ( Quitter);
JMenu menuEdition = new JMenu(" Edition ");
JMenuItem Couper=new JMenuItem("Couper"); menuEdition.add ( Couper);
JMenuItem Copier=new JMenuItem("Copier"); menuEdition.add ( Copier);
JMenuItem Coller=new JMenuItem("Coller"); menuEdition.add ( Coller);
JMenuBar barreMenu = new JMenuBar ();
barreMenu.add( menuFichier);
barreMenu.add( menuEdition);
fen.setJMenuBar( barreMenu);
```



IHM - Java

Eléments de base d'une IHM Java

(CHAPITRE 4)



Les dialogues de sélection de fichiers : JFileChooser :

Comme toutes les interfaces graphiques, Swing propose une boîte de sélection de fichier(s) avec la classe **JFileChooser**. La classe propose des méthodes pour afficher un dialogue d'ouverture de fichier.

- **showOpenDialog** présente une boîte de dialogue pour l'ouverture d'un fichier
- **showSaveDialog** pour la sauvegarde d'un fichier

Contrairement aux boîtes de dialogues, un objet **fen** doit être instancié :

```
JFileChooser fc = new JFileChooser();
fc.showOpenDialog(fen);
```

La méthode `getSelectedFile` renseigne sur le nom du fichier sélectionné.

```
if (fc.showOpenDialog(fen)== JFileChooser.APPROVE_OPTION){
    System.out.println("Le fichier est:"+fc.getSelectedFile());
}
```

IHM - Java

54

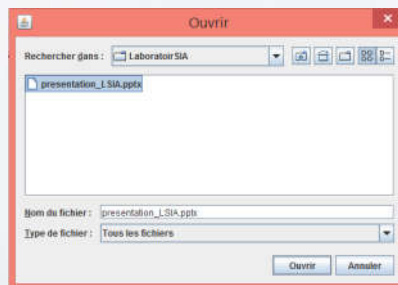
Éléments de base d'une IHM Java

(CHAPITRE 4)



Les dialogues de sélection de fichiers : JFileChooser :

```
JFrame fen = new JFrame ("Ouvrir");
JFileChooser fc = new JFileChooser();
fc.showOpenDialog(fen);
if (fc.showOpenDialog( fen )== JFileChooser.APPROVE_OPTION){
    System.out.println("Le fichier est:"+fc.getSelectedFile());}
```



Problems Javadoc Declaration Console

```
Test (1) [Java Application] C:\Program Files\Java\jre1.8.0_60\bin\javaw.exe (18 févr. 2016 09:55:11)
Le fichier est:E:\LaboratoireSIA\presentation_LSI.pptx
```

IHM - Java

55

Éléments de base d'une IHM Java

(CHAPITRE 4)



Les composants orientés texte :

- Swing propose des composants de écriture de texte :
 - **JTextField**
 - **JFormattedTextField**
 - **JPasswordField**
 - **JTextArea**
- Tous ces composants descendent (directement ou non) de **JTextComponent**.

IHM - Java

56

Éléments de base d'une IHM Java

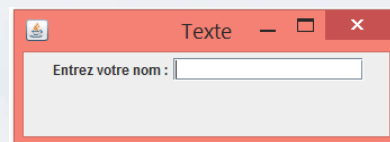
(CHAPITRE 4)



Les composants orientés texte :

Pour saisir une seule ligne de texte, on utilise le composant **JTextField**. Il construit un champ de saisie dont la largeur peut être fixée avec **setColumns**. Il est préférable de fixer la largeur du champ de saisie pour éviter des déformations des interfaces graphiques lors du remplissage.

```
JPanel pan = new JPanel();
JLabel lNom = new JLabel ("Entrez votre nom :");
JTextField tfNom = new JTextField();
tfNom.setColumns(15);
pan.add (lNom);
pan.add (tfNom);
fen.setContentPane(pan);
```



IHM - Java

57

Éléments de base d'une IHM Java

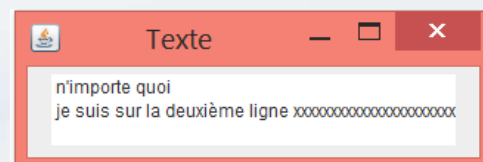
(CHAPITRE 4)



La zone de texte JTextArea :

- Afficher un texte sur plusieurs lignes.
- Le texte est affiché en bloc, avec une police unique. Pour remplir le composant on utilise la méthode **setText**

```
JFrame fen = new JFrame ("Texte");
JPanel pan = new JPanel ();
JTextArea taNotes = new JTextArea();
taNotes.setText(" n'importe quoi \n je suis sur la deuxième ligne
xxxxxxxxxxxxxxxxxxxxxxxx\n");
taNotes.setEditable(false);
pan.add(taNotes);
fen.setContentPane(pan);
```



IHM - Java

58

Éléments de base d'une IHM Java

(CHAPITRE 4)



Exercice :

Désignation	Prix unitaire	Quantité	Prix total
PC Pentium III	10000	2	24120
Ecran 17"	2500	2	6030
Imprimante laser	4500	1	5427
Scanner	990	1	1193,94
Manuel d'utilisation	500	1	527,5

- Concevoir l'IG de cette facture
- Programmer le bouton «**Nouvelle facture**» de manière à vider les listes d'édition qui composent la facture, à remettre le montant total à zéro.
- Programmer le bouton «**Ajouter à la facture**», puis vérifier vos calculs avec le jeu de données ci contre
- Le taux de TVA peut être normal (20 %) ou réduit (5,5 %).

59